Golden Thread Plugin

Golden Thread Plugin — Documentation

> **Reader:** quick lookup, and the printed PDF **Claims last checked against the code:** 2026-10-02 (gt 0.19.1) — see *The documents, and what belongs in each* in `CLAUDE.md` .

Version gt 0.21.4 / gt-wiki 0.2.9 / gt-usage 0.1.10 / gt-visualize 0.4.6 / gt-lotr 0.5.0 / gt-pane 0.1.1 / gt-workflow 0.1.0 / gt-farm 0.21.1 / gt-report-card 0.21.1 / gt-demo 0.21.4 / gt-watch, gt-flow 0.21.0

Golden Thread turns an Obsidian vault into the single source of truth for all AI memory across every project and every session. The tiered rule model introduced in v0.6.0 now carries **ten hook-backed Core rules** enforced at three points in the turn, 0.9.12 added `gt-route` for the middle of a session, and 0.9.13 makes the session-start component check verify that the hooks are **wired**, not merely installed, 0.11.0 makes `log.md` and `decisions.md` generated files so concurrent sessions cannot overwrite one another, 0.12.3 stops a vault tool running against a vault it was never told to touch, 0.12.4 makes parallel execution the default for divisible work in every project, 0.12.5 stops code being committed before its tests have been seen to pass, 0.13.0 makes an upgrade from any older release end where a fresh install would, 0.14.0 splits optional parts into modules you can decline, and 0.15.0 makes six of them — gt-wiki 0.2.1 (an LLM-powered knowledge base with immutable sources and interlinked pages), gt-demo (the guided tour), gt-watch (upstream repos), gt-report-card (the session report card), gt-farm (work packets for an external AI) and the new gt-flow, which draws the vault's event stream as a timeline of knowledge climbing the ladder. 0.16.0 gave contributed **packs** — plain JSON data, reviewed and merged, never third-party code — somewhere to come from and something to read them, shipping `gt-scan` , `gt-optimize` , `gt-handoff` , `gt-allin` and `gt-allin-commit` ; 0.16.1 added `gt-context` (the first consumer of the registry's model-reachable tier) and `gt-validation` (a verification receipt that goes stale when the file it describes changes); and 0.16.2 re-asserts the Core rules after a compaction as well as on every prompt, lets a module hook declare a

`timeout`, and refuses a copyleft licence by its current SPDX spelling; 0.16.3 makes `install.sh` refuse when MANIFEST.json is tracked but missing rather than regenerating one, and stops `gt_tasks.py` overwriting TASKS.md in silence. 0.17.2 adds `gt_surface.py`, a `SessionStart` hook that finally reads what earlier sessions wrote for the next one — a live MUST DO block from `deadlines.md`, every handoff not yet handled, a count of urgent tasks waiting on you, pre-compaction state — adds five skills that create, list and handle tasks and handoffs (`gt-task`, `gt-task-list`, `gt-task-handle`, `gt-handoff-list`, `gt-handoff-handle`), and keys session claims on a machine id rather than a hostname. 0.17.11 made Core rule 1 queue-first. 0.18.1 makes the skills one verb per action — `gt-create`, `gt-open`, `gt-list`, `gt-handle` and the new `gt-close`, each taking the artifact as its argument, with the six old task and handoff names kept as deprecated aliases until 0.21.4 removed them — and adds `gt-plan` and `gt-implement` for the coding loop, `gt-brief` to draft a repo's `CLAUDE.md` from the vault, and `gt-minimize` to prune a session before cutting it. 0.20.1 adds **gt unlock**, off by default: agents need your presence (TOTP plus Touch ID or Windows Hello, optionally Entra ID) before they use LOTR connections, credentials, publishing or gt's guards — see the security section below and `SECURITY.md`. 0.21.0 ships hooks with the plugins (a `hooks/hooks.json` in gt and four modules, dispatching through `~/.claude/golden-thread/wired/`, used from Claude Code 2.1.136), makes the install **ask every choice** with the security level first (`install.sh --questions`, exit 5 when a security answer is missing off-terminal), adds the `paranoid` group with single-use action tokens, I/O brokers for disk read, network fetch and disk write, a token profile, a models-and-cost page with a calibration schedule, the optional **gt-pane** module, `/gt:gt-help`, `/gt:gt-complete` (**breaking:** `/gt:gt-close` now closes the session), `/gt:gt-forget`, `/gt:gt-grant` and `/gt:gt-calibrate`. 0.21.4 connects an MCP service in a few steps (`lotr setup`, gt-lotr 0.5.0, with a catalog of sixteen templates released apart from gt in `lotr-templates/`), lets a persona shape the session (`featured_skills`, `session_focus`, `lotr_templates`), adds a First day walkthrough and a guide per role to `/gt:gt-help`, `install.sh --only NAME` for one module, and makes `~/GoldenThread` the default vault, with a warning for a vault in Documents or on a synced or shared drive.

What It Does

Instead of scattered `.claude/memory/` files and CLAUDE.md snippets that live and die per session, Golden Thread gives every fact a permanent home in a structured vault. Knowledge flows up a hierarchy from session notes into the vault, and the right facts are always in scope when you need them.

```
session conversation
      ↓  gt-work / gt-promote
project memory  (memory/*.md)
      ↓  gt-promote
project files   (research.md / decisions.md / design.md / spec.md / runbook.md)
      ↓  gt-promote
Knowledge/      (cross-project wiki pages, backed by immutable Sources/)
      ↓  gt-promote
global-memory/  (loaded in every session, all projects)
```

Facts move up the hierarchy as they prove themselves general. They never move back down and are never silently deleted.

Core Rules (gt 0.17.2)

Golden Thread defines a tiered rule model that separates rules by scope and enforcement strength. **Ten Core rules ship as of 0.15.0** and remain ten in 0.17.2, up from one at 0.6.0 (the count read "seven" from 0.9.10 through 0.12.3, one behind the files):

#	Rule
1	Write vault content only through the write queue (`gt_write_queue.py`), then apply it with `gt_broker.py drain`; never edit a vault file directly, and never write one another live session has claimed. (Revised in 0.17.11: was "claim, then write"; the rule id `core_concurrent_session_claim` is unchanged.)
2	Name the vault on every mutating tool run — `--vault` or `--dry-run` — never let the target be inferred.
3	Never put a secret's value into the session — not to inspect it, not to redact it, not to check it.
4	Run the tests before you commit code, and say which ran — never commit code whose tests you have not seen pass.
5	Begin every response with the current wall-clock timestamp — before any other text you emit.

#	Rule
6	`global-memory/` contains only facts needed in EVERY project.
7	Do not auto-load the full memory index.
8	Parallelise any work that can be parallelised, in every project — independent units run concurrently within one machine-wide budget shared by every session, and serial execution must be justified, not assumed. (Reworded in 0.15.0: the `parallel_max` budget is one for the whole machine, not one per session.)
9	A secret's value rests only in the secrets store or a mode-600 file the store wrote — never in source, a vault file, a repo, a log, or a session.
10	Label every derived figure you present as fact — and every file path you assert or hand to the user, after checking it (`gt_path_check.py` , or `path_stat` where the directory can't be read) — with its verification state: `unverified` , `self-verified` , or `independently verified` .

The numbers are the order the `UserPromptSubmit` hook injects them in. Verify the set at any time with `echo '{}' | ~/.claude/golden-thread/hooks/inject_core_rules.sh` .

Since 0.16.2 the same script is registered a **second time**, on `SessionStart` with a `compact` matcher, because a compaction does not preserve what a hook added earlier: project-root `CLAUDE.md` and auto memory are re-injected from disk, while hook-added context is summarized with the rest of the conversation. The exposure this closes is narrow and worth stating precisely — `UserPromptSubmit` has no compaction exception, so the next user prompt always restored the rules verbatim; what was exposed was the *remainder of the turn* in which an auto-compaction fired. The mechanical tier was never affected: the `PreToolUse` guards and the `Stop` validator are event-driven commands, not context. What lapsed was the re-assertion, not the backstop.

Three scope levels:

Level	Meaning
Core	Applies to every chat, every project, no exceptions. Un-removable.
Context	Applies to this project or session type only.
Generic	Default; no special standing.

Two enforcement strengths:

Strength	Meaning
Reminder	Injected on every turn via `UserPromptSubmit` hook.
Validated	Checked by a `Stop` hook that blocks any reply violating it; machine-enforced.

Since 0.12.3 a **second `PreToolUse` guard** (`guard_vault_writes.sh`) denies a vault-mutating tool run that does not say *which* vault it means — no `--vault`, no `--dry-run`, no `GT_VAULT`. It exists because a session rehearsing the 0.11.0 migration in a scratch copy had one tool that took `--vault` and one that did not, so the same loop migrated all 42 `decisions.md` files of the live vault. A rehearsal that cannot be told where to rehearse is not a rehearsal. Read-only subcommands are never denied, and anything it cannot parse with certainty is allowed — it fails open, as every guard here does, because a guard that blocks wrongly makes every session unusable.

Since 0.9.5 a third point exists: a `PreToolUse` hook (`guard_session_claims.sh`) denies a `Write` / `Edit` to a vault file another live session holds a claim on. **Since 0.17.11 it enforces queue-first:** a direct `Write` / `Edit` to any vault `.md` outside `Sources/`, `core-rules/`, `.obsidian/`, `.git/`, `.gt/` and gt's own `spool/`, `sessions/` and `tools/` is denied, claimed or not, with the exact queue command in the reason; so are the visible shell writes into the vault (`>`, `>>`, `tee`, `sed -i`, `cp` / `mv`). A script that opens a file itself is not visible to it, and the rule says so. It fails open on any parse failure.

Core/Validated rules are hook-backed. Hook scripts install to `~/.claude/golden-thread/hooks/` during `install.sh`. `settings.json` references them by absolute path so they survive vault renames and project moves. `gt_paths.py` is a self-healing resolver — if a recorded path is stale, it locates `core-rules/` by scanning the vault rather than failing silently.

The canonical rule definitions live in `Projects/golden-thread/core-rules/` inside the vault. Editing a rule file there changes what the hook injects — the rule text is never duplicated into the script.

The canary: `core_timestamp_every_message.md` is designated Core/Validated not because a missing timestamp causes real harm, but because it is trivially observable at the top of every reply. When the timestamp disappears, enforcement has broken.

---

gt Skills (37)

Setup

Command	What it does
`/gt:gt-setup`	Finish an install from the Claude prompt: everything / choose modules / core only, then the security level and the vault, then the installer runs with your answers. Run it after `/plugin install gt-all@golden-thread-plugin`.
`/gt:gt-init`	Create a new vault from scratch, or wire an existing vault to a project. Idempotent — safe to re-run.
`/gt:gt-create`	The create verb (0.18.1): `project <slug>` (or a bare slug) scaffolds a project folder with the standard structure and captures your brain dump into `idea.md` — sub-projects, tags, runbook; `task <text>` adds a task line through `gt_task.py add`; `handoff` writes a handoff through `gt_handoff.py`.

Daily Work

Command	What it does
`/gt:gt-open`	Load a project at session start. Reads the core project docs in order — `source.md` first, so you know which host serves which role before touching code, then idea → research → decisions → design — and *indexes* the memory files rather than loading them. Names the project's waiting handoffs (`gt_surface.py handoffs --project`) without reading them. Since 0.18.1: a catch-up brief after `brief_absence_days` away (`gt_catchup.py`; `--brief`, `--no-brief`), `research-digest.md` in place of research headings when current (`gt_digest.py check`), a line on which repo a repo-scoped command will hit, and `handoff [id]` / `task <id>` to open one of those read-only.
`/gt:gt-route`	Mid-session. Names what the session has actually become, says where its output belongs, and checks you are in the right project, harness and model. For when a session drifted from what it opened with, or you cannot name what you are doing.
`/gt:gt-work`	Write back session findings. Appends to `research.md`, adds ADRs to `decisions.md`, refines `design.md`, creates `spec.md` when design is complete, and flags content for PROTOCOL.md. Offers a handoff for what did not reach a file; each task it raises names the handoff's filename. Since 0.18.1: asks about

Command	What it does
	ADR `--supersedes` / `--expires-when` and memory `entities:`; regenerates `research-digest.md` (`gt_digest.py write`); runs `gt_memory_check.py` (contradictions) and `gt_promote_detect.py` (promotion candidates); a *Learn* step offers reusable patterns; the skeptic pass needs only `skeptic_pass` .
`/gt:gt-ingest`	Bulk-import an existing project's memory files, CLAUDE.md rules, and notes into the vault. External sources are stored immutably in `Sources/` before being synthesized into Knowledge pages. Resumable from any session since 0.18.1 (`--resume CK` , `--done CK --index N`).
`/gt:gt-review`	Empty the inbox: sweep `INBOX.md` — plus Obsidian daily notes if you keep them — for captured-but-unfiled thoughts and route each one into a tracked project.
`/gt:gt-sync`	Vault ↔ its git remote (0.18.1, `gt_sync.py`): `status` (ahead/behind/uncommitted), `pull` (fast-forward only; stops on divergence), `push` (push check must pass; refused while behind). `sync_check` (`off` / `cached` / `fetch`) adds a behind line at session start.

Coding Work (0.18.1)

Command	What it does
`/gt:gt-plan`	Restate the requirement and confirm it, read the design/spec/ADRs and code it touches, list risks verified or unverified, write a phased test-first plan with a strict `scope:` block (branch, allowed files, generated files, new dependencies) and a `## Deviations` section to the repo's git directory (`.git/gt/plan-current.md` ; `status: draft`), and stop at "Approve this plan?". An explicit yes runs `gt_scope.py approve` . No code.
`/gt:gt-implement`	Refuse without an approved plan; then per phase: failing test, code, refactor, `gt_allin.py --only tests,scan` naming the tests that ran, then `gt_scope.py check` against the plan's scope contract (every deviation shown with its written reason); stop on any failure or any check that could not run; commit only via `gt_allin_commit.py` on your yes; never push.

Command	What it does
`/gt:gt-digest <file> [what you want]`	Have a big file, a held-back paste or an image read by the cheapest reader and get back only the answer: a local engine (gt-farm, secret-scanned first), else an Explore subagent when the file is larger than a helper costs to start (`gt_break_even.py`), else read inline. `gt_intake.py plan` decides; the file never enters this conversation unless it is small.
`/gt:gt-minimize`	Prune a heavy session before cutting: `gt_minimize.py` measures billed context and cache warmth; keep what matters (Knowledge/INBOX through the queue), drop the rest, then "/compact or /clear now". No in-flight note — that is `/gt:gt-create handoff` .

Tasks and Handoffs (0.18.1: one verb per action)

Command	What it does
`/gt:gt-create task <text>`	One well-formed line in the project README's `## Tasks` via the vault tool `gt_task.py add` , with `[p::] [waiting::] [since::]` filled in and an optional `[ref:: …]` to a `[[page]]` or vault path that must resolve. `--inbox` for unfiled. Refuses a README another live session has claimed.
`/gt:gt-create handoff`	Write the next session a handoff it can trust: facts from the project and repository, each labelled with source and verification state, plus *What Changed This Session* (`git diff <start_commit>..HEAD`). The design narrative is left for the session that did the work.
`/gt:gt-open task <id>` · `handoff [id]`	Read-only: one task's full detail, or one handoff and only the task lines citing it.
`/gt:gt-list`	Read-only. `tasks [filter]` — `<slug>` / `inbox` , `p1` `p2` `p3` , `mine` , `overdue` , `stale` , `deferred` , `ref:<text>` ; each line carries an ID, `slug:LINE:HASH` . `handoffs` — open or deferral ended, one line each, `--all` for every state. Bare: a summary of both.
`/gt:gt-handle`	One at a time. `task [filter]` : close (`gt_close.py task`), drop (reason), defer (future date), move (`gt_task.py move --to`), or keep; an ID whose line

Command	What it does
	changed is refused. `handoff`: settle each item, then close as `handled` (`gt_close.py handoff`) or defer to a date. Bare: asks which.
`/gt:gt-complete`	`project <slug>`: every open task and handoff decided (complete / drop / move / keep shelved at `p:: 7`; `gt_close.py project` exits 1 until then), graduation offered through gt-promote, then archived in place (`stage: archived`); `--move` relocates to `Archive/<slug>/`. `task <id>`; `handoff [id]` (since 0.21.0 only when each open item is resolved; refused while a citing task is open). **Breaking in 0.21.0:** this was `/gt:gt-close`.
`/gt:gt-close`	*(0.21.0)* Close this session, the opposite of `/gt:gt-open`: write back what it learned and release its claim.
`/gt:gt-calibrate`	*(0.21.0)* Run the model calibration pass now (alias for `gt-optimize --only calibration`); `gt_model_calibrate.py plan --estimate` prices it first.
`/gt:gt-forget`	*(0.21.0)* Retract a wrong fact in every note that states it; `gt_forget.py search|retract`; nothing is deleted.
`/gt:gt-grant`	*(0.21.0)* Let one command through for a while without lowering the lockdown level.
`/gt:gt-help`	*(0.21.0)* An offline page that plays a command's real output in a simulated terminal.

Knowledge Management

Command	What it does
`/gt:gt-query`	Look something up — reads `index.md` first, follows wikilinks, falls back to grep. The vault is always checked before the web. `--lineage <topic>`: the supersession chain of ADRs about it (`gt_adr.py lineage`, 0.18.1). `--entity <name>`: only the memory notes declaring it (`gt_entities.py`, 0.18.1).
`/gt:gt-promote`	Graduate a fact up the hierarchy: project memory → project files → Knowledge wiki page → global-memory. Also handles new project scaffolding and retiring stale content.

Command	What it does
`/gt:gt-brief`	Draft a self-contained `CLAUDE.md` section for a project's code repo — the project, constraints from current ADRs, where it runs, what not to do — printed, never written (`gt_brief.py --vault V <slug> [--repo PATH]`, 0.18.1). The skill places it through the queue on your yes.
`/gt:gt-refresh`	Check `Sources/` for upstream changes. Supersedes outdated sources with new immutable files — never edits the old one. Updates Knowledge pages that cited the changed source.

The same ladder downward is a script, not a skill. `gt_demote.py --vault V --file <path> [--to knowledge|project-memory] [--project <slug>] [--apply]` ranks by **cost**, not by maturity: `global-memory/` is read in every session of every project, `Projects/<slug>/memory/` in every session of one, `Knowledge/<page>.md` only when someone asks. A fact in global-memory that one project needs is charged to every session for ever; moving it makes it cost what it is worth. Dry run unless `--apply`. It writes the destination, verifies it by re-reading from disk, and only then removes the source, leaving a pointer — so a failure leaves **duplication**, which a reader can resolve, rather than the deletion that got `gt_optimize --apply` removed. Refuses `core-rules/`, `Sources/`, and a file another live session has claimed; a session file it cannot read refuses too, because "could not check" is not "clear".

Context & Verification

Command	What it does
`/gt:gt-validate`	Verify a claim by re-deriving it with a fresh-context validator — never by reviewing the reasoning that produced it. Use before recording a finding as fact or before a production change. Since 0.18.1: `gt_check.py list --for` first; `model_intent: deep`.

Maintenance

Command	What it does
`/gt:gt-upgrade`	Bring an existing vault up to the installed release: run the migrations it has not had, take the release's changes into `PROTOCOL.md` and `CONVENTIONS.md` without

Command	What it does
	losing local edits, add newly shipped Core rules, stamp the vault. Rehearse with `--dry-run`; it refuses a dirty tree and backs up before applying.
`/gt:gt-doctor`	One report for the whole install: plugin version, component drift, hook wiring, the vault's release stamp and pending migrations (asked of `gt_upgrade` itself since 0.17.2), the last exit of every installed scheduled job (`schedule`, 0.17.2), stray workers, unpushed commits, publish-destination drift, a lint summary, and since 0.18.1 `repo-target` (which repo a repo-scoped command resolves to — a note, `i`, never a finding) and `hooks-schema` (`settings.json` hook entries naming an unknown event or tool, or a missing gt hook file). Exit 2 means a check *could not run*, which is deliberately distinct from clean.
`/gt:gt-lint`	Audit the vault for structural problems: gt_lint runs 30 checks covering broken wikilinks, orphaned pages, missing index entries, unlisted memory files, Knowledge pages citing superseded sources, stale pages, and `core-unenforced` — a Core rule that is stored but wired to no hook. Since 0.18.1 four file questions to the review queue: `adr-expires`, `bundled-concept`, `decision-candidate` (phrases: setting `decision_signals`), `memory-entity-orphan`; and `release-pipeline` checks each project's `release_pipeline:` flag against its code.
`/gt:gt-optimize`	Two members (0.18.1). `vault`: a fact duplicated across memory files, a dead index row, a `global-memory/` file over budget or naming one project (`single-project-global`), a relative date, a Knowledge page unread for 90 days (`knowledge-unused`, from the `log_knowledge_read.sh` read log), and `--cost` per project. `session` (`gt_optimize_session.py`): prompt-cache writes by cause — cold, growth, expiry, invalid — with list-price equivalents. Reporting never writes; `--demote`, `--archive --project S --before D` and `--supersede` write only with `--apply`, through the queue, and delete nothing. Exit 3 when a member could not run.

Command	What it does
`/gt:gt-scan`	Scan code against the language definitions in effect on this machine — naming conventions and encoding, per language, all of it from packs rather than from the script (fourteen languages ship with definitions; `gt_scan_language.py --languages` lists what is in effect here). An aggregator over leaf scanners: it reports how many members RAN alongside what they found, because "nothing is wrong" and "nothing was checked" otherwise print identically. Resumable since 0.18.1 (`gt_scan.py --resume CHECKPOINT`).
`/gt:gt-allin`	Run every check in one command — scan, lint, the optimize report, install health, and since 0.17.2 `secrets` (`gt_secrets.py` over `--repo`), `runbooks` (`gt_lint.py --runbooks`), `tests` (the repo's own suite, recording the receipt `/gt:gt-allin-commit` checks), `validations` (`gt_validation.py list`) and `wiki` (gt-wiki's `wiki_lint.py` , only while that module is installed) — and report how many members actually RAN alongside what they found, because a short finding list from a half-failed run reads exactly like a clean bill of health. Never pushes and never applies a change: `--suggest-push` prints the command for you, and refuses even that when anything failed.
`/gt:gt-context`	Render the definitions this vault marks as model-reachable — `vocabulary` , `validation_rules` , `runbook` — inside an explicit untrusted-data envelope, hard-capped, with every line naming the pack and tier it came from. The envelope is **not a security control** — no in-context framing is: *Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents* (NAACL 2025 Findings, arXiv:2503.00061) attacked eight published defences and broke all eight, above 50% attack success in every case. What the envelope provides is *identifiability*: it marks content as someone's definition rather than as the system speaking. What protects a session is that packs are reviewed before merge, and that a pack can only reach a session already trusting the vault. Asking for a Tier A slot is a usage error, not an empty section.
`/gt:gt-validation`	Record what a validation established about a file — what was verified AND what it could not determine — stamped with the file's content hash, so the recorded definition goes visibly stale the moment the file changes. A file with no receipt reports as *unknown*, deliberately distinct from clean.
`/gt:gt-allin-`	Commit, but only once the checks pass and a passing test receipt covers every staged file. Refuses on `main` / `master` without a flag, refuses when a

Command	What it does
`commit`	check *could not run* (an unknown is not a finding anyone can accept), and never pushes — a commit is reversible with `git reset`, a push is fetched by other people. It checks the receipt itself because `guard_test_before_commit` is a PreToolUse hook and cannot see a script running git.
`/gt:gt-runbook-lint`	Scan all project `runbook.md` files for content that has drifted into multiple runbooks. Routes duplicated content to the right shared layer via `gt-promote`.
`/gt:gt-settings`	View and change what Golden Thread does on its own. gt's own settings are `component_updates`, `version_check`, `orphan_check`, `push_check`, `surface`, `daily_comms_content`, `daily_finalize`, `release_announce`, `agent_specialization`, `skeptic_pass`, `handoff_surface`, `task_surface`, `test_gate`, `parallel_work`, `parallel_max`, `protected_paths`, and new in 0.18.1 `brief_absence_days`, `foreign_checkout_guard`, `decision_signals`, `knowledge_access_log`, `optimize_session_days`, `optimize_avoidable_pct`, `memory_contradiction_check`, `promotion_candidates`, `promotion_overlap`, `review_stamp`, `sync_check`, `reminder_days`, `reminder_macos`, `reminder_relay`, `reminder_email`, `commit_checks`, `addon_fixes`, `addon_fix_size_limit`, `execution_metrics`, `scoped_receipts`, `test_tmpdir` and `runners`; each installed module registers its own from `module.json` and they are listed under the module's name (`report_card`, `closeout_check`, `usage_meter`, `usage_alert`, `visualize_publish`, `visualize_publish_visibility`, `watch`). Every automatic behaviour is registered here and every one can be switched off.

Modules (9)

Optional parts, each a separate plugin in the `golden-thread-plugin` marketplace, declared by a `module.json`, versioned with gt and installed by `install.sh` while on. `bash install.sh --list-modules` shows each one's state; `--without <name>` / `--with <name>` change it and the choice is remembered. A module's hooks are wired only while it is on and are always *reporter* hooks — a Core-rule enforcement hook can never belong to a module. A module hook may declare an optional `timeout` in whole seconds (1–600); `report-card` declares 15 on `SessionEnd`, because `SessionEnd` hooks share a 1.5-second budget by default and one that overruns is cancelled with its **output discarded**, so a report card could have started truncating invisibly. Module settings

(with an optional long `detail`) register from `module.json` and are listed under the module's name in `/gt:gt-settings`; a value set for a module that is later switched off is kept.

Command	Module (plugin) · default	What it does
`/gt-wiki:gt-wiki` and four more	`wiki` (`gt-wiki`) · on	A standalone LLM wiki: immutable sources, interlinked pages, ingest, lint and refresh — see below.
`/gt-demo:gt-demo`	`demo` (`gt-demo`) · on	A guided tour on the fictional PizzaBot 3000 project, run in its own throwaway vault: sixteen core acts (Core rules, open, task rollup, capture, route, validate, lint, promote; then 0.21.4's fewer tokens per turn, long output and big pastes, compact before it costs you, notes answered locally, what your own engine is good for, check the path, right model and effort; inbox and close) plus one act from each installed module that ships one — `wiki`, `watch` and `flow` by default, `workflow` and `lotr` when installed. Every 0.21.4 act runs real gt commands offline; what cannot run offline is shown as its setting and labelled a description. `start`, `tour`, `end`, `clean`, `remove`, `status`.
`/gt-watch:gt-watch`	`watch` (`gt-watch`) · on	Watch any git repo. `add <url>` writes a watch note; a cron fetch (`gt_watch.py fetch`) classifies every change by rules — P0 for security advisories, CVE/GHSA ids or security releases; review for new releases, major bumps, changes under `watch_paths`; routine otherwise — and a SessionStart hook reports unacknowledged changes, P0 first. `show` explains a change, `ack` marks it seen. Setting `watch`, default `off`. `--without watch` also removes its crontab line. Was `/gt:gt-watch`.
(no command)	`report-card` (`gt-report-card`) · on	The session report card at `/compact` and session end — saved as a notice and shown at the next session start, because output at those events is never displayed — and the project close-out question. Settings `report_card`, `closeout_check`.

Command	Module (plugin) · default	What it does
`/gt-farm:gt-farm`	`farm` (`gt-farm`) · **off** fresh, on for upgraders	Route bulk, mechanical, or second-opinion tasks to an external AI service as a self-contained work packet. All four gates (Stateless, Self-contained, Checkable, Releasable) must pass before a task leaves. Results come back unverified. Was `/gt:gt-farm`; a machine upgrading from a gt that shipped it keeps it on.
`/gt-workflow:gt-workflow`	`workflow` (`gt-workflow`) 0.1.0 · **off**	New in 0.21.4. Workflows you own: a saved chain of steps (a pipeline is the same thing) run by name, `/gt-workflow NAME [values]`. Steps: `run` (a gt tool from an allow-list), `lotr` (read on its own; write only after you approve that exact step; consent never), `farm` (a local engine, handed to Claude when unsure), `claude`, `agent`, `you`. Inputs are typed and checked first; `when:` branches; outputs pass by name. `new --from-description` previews a draft and saves only on `--save`; `schedule` installs a `workflow-NAME` job for the ones that need nobody. 15 starters.
`/gt-flow:gt-flow`	`flow` (`gt-flow`) · on	New in 0.15.0. Renders `events.jsonl` as one offline HTML file: a lane per project, time left to right, an arrow each time an item climbed a level. `--redact` hashes every name before the page is shared; task events are hidden until `--tasks` or a click; `--project`, `--since`. Never writes the vault.
`/gt-visualize:gt-visualize`	`visualize` (`gt-visualize`) 0.4.1 · on	New in gt-visualize 0.4.0. Two modes, each one offline HTML file with three.js inlined. **explain**: a scroll-driven walkthrough of how a codebase's parts work together — scenes highlight parts and animate the flows between them, from a story Claude writes out of the code and the vault (`--check` validates it). **render**, the code city: directories are districts, files are buildings — height by lines, footprint by size, colour by language (gt's filetype definitions) or by git churn over `--since` days. Orbit, hover, click to focus, search. `--redact` hashes every name before sharing; an `--out` inside the vault is refused.

Command	Module (plugin) · default	What it does
`/gt-lotr:gt-lotr`	`lotr` (`gt-lotr`) 0.1.0 · **off**	New in 0.17.11. LOTR, also called gt MCP: one gateway to rule them all. Four fixed MCP tools — `find` , `call_read` , `call_write` , `call_consent` — in front of any number of downstream connections (profiles `github` , `jira-v3` , `jira-v2` , `graph` , `generic`), with a registry of who talks to whom, as whom, from which machine. CLI `lotr` (also `mcp`): `find` , `read` / `write` / `consent CONN OP [k=v …] [--select] [--cursor]` ; admin on the registry machine only: `--zone Z init --mode local|hub|client` , `add-http` , `enroll` , `revoke` , `status` . Daemon `lotrd.py` ; stdio shim `lotr_mcp.py` . A `wrong_tool` error means the gateway classed the op higher; results are `untrusted` ; credential-shaped text is `withheld` . Credentials are references (`keychain:` , `store:` , `file:`), never values. Hybrid placement is parsed and refused. Agents (lotr 0.4.0, gt 0.20.2): `gt-lotr:jira` , `gt-lotr:m365` , `gt-lotr:github` , `gt-lotr:runner` take bulk and multi-call work (sweeps, triage, paging) over the session's connection and return ≤ 15 lines (`ANSWER` , `ITEMS` , `MORE next_cursor` , `CALLS` , `GAPS`); haiku via `model_intent: fast` (`gt_model_policy.py set --agent gt-lotr:<name>`); a bulk write runs on the definition's policy-written `writes_model:` (sonnet; `--writes-model` per agent). 0.5.0: `lotr setup TEMPLATE [--id name@zone] [--param N=V] [--yes]` (check, runtime, credential, add, smoke test; exit 3 = waiting for you), `lotr secret put NAME` (hidden prompt, own terminal), `lotr template new --archetype A` , `template facts-check` , `template update [--ref R] [--dry-run]` (fetch the separately released template catalog; creates no connection), `add-mcp --from-template` (kind `mcp-http` , key as bearer or named header), `connect --client-id-ref/--client-secret-ref` . Turn on with `install.sh --with lotr` .
`/gt-lotr:gt-lotr-research`	`lotr` (`gt-lotr`) 0.5.0 · **off**	New in gt-lotr 0.5.0. The research run: given a service name, reads the vendor's own documentation, picks one of nine template archetypes, fills the template and writes a fact

Command	Module (plugin) · default	What it does
		sheet that records, for every fact, its source URL and verification state (`unverified` , `self-verified` , `independently-verified`). Never handles a credential; never connects to the service. Output: `NAME.json` and `NAME.facts.json` , checked by `lotr template check` and `lotr template facts-check` . The execution run is `lotr setup NAME` .
`/gt-usage:gt-usage`	`usage` (`gt-usage`) 0.1.3 · on	New. Reports the 5-hour, weekly and monthly-spend windows from the readings its status line records, and what ending or cutting a session would save. A window the plan does not report is shown as absent, never as 0%. Settings: `usage_meter` , `usage_alert` . Never writes the vault.

Movement events (0.15.0)

`gt_events.py` (schema v1, since 0.13.0) now receives events from the operations themselves: `gt_log.py add … --event <kind> --item <path>` spools a log line and its event in one command (used by `gt-promote` and `gt-refresh`); `gt-review` , `gt-work` and `gt-ingest` call `gt_events.py emit` ; `gt_adr.py allocate` emits `adr` ; `vault_init.py` emits `create` , `rename` , `merge` and `archive` (the only source of `archive`); `gt_tasks.py` emits `task.open` / `task.done` once the stream has been seeded. An event can never fail the operation it records. A vault older than the events recovers its history with `gt_events.py --vault <vault> backfill --dry-run` (then without `--dry-run`): git and `log.md` rebuilt as `actor: backfill` events, idempotent.

Concurrent sessions stop colliding (0.12.3)

`log.md` and `decisions.md` are the two files every session appends and none owns. In one working tree there are no branches to collide and no merge to resolve — just last-writer-wins, invisible when it happens.

Both are now **generated**, joining `TASKS.md` :

File	Written by	Rendered by
`log.md`	`gt_log.py add` — one spool file per session	`gt_log.py merge`

File	Written by	Rendered by
`decisions.md`	`gt_adr.py allocate <project>` — one file per ADR	`gt_adr.py merge <project>`

No session writes a shared file, so there is nothing to claim and nothing to guard, and `git add` scopes to your own work by construction rather than by discipline.

ADR numbers are the harder half, because a number is a scarce identity — two sessions may both write a log line in the same instant, but they cannot both be ADR-6. A counter that is read and then written back does not fix that: read-then-write is two operations, so both sessions read 5, both write 6, and both use ADR-6. `gt_adr.py allocate` instead creates the slot with a single atomic exclusive create, so the filesystem decides the winner. Verified under contention: 100 racing process pairs, 200 allocations, no duplicates.

Migration (`gt_log.py migrate`, `gt_adr.py migrate <project>`) freezes the existing file as a baseline that sorts first — no history is rewritten and nothing is renumbered — and refuses unless the merge reproduces the original byte for byte. It also refuses on a project whose ADR numbers already collide, because renumbering would invalidate inbound references; `gt-lint`'s `adr-collision` check reports those for an owner to resolve.

Which machine wrote a claim (0.17.2). `gt_session.py` claims files for a session, and a claim counts only while its session is live — judged by the recorded pid, which means something only on the machine that recorded it. That machine is now a uuid created once in `~/.claude/golden-thread/machine-id`, not the hostname: on 2026-09-28 a DHCP rename made every claim written under the old name unjudgeable, and Core rule 1 lapsed in silence. A rename is reported once; a file from an older gt with no id keeps its claims while its host label matches, and otherwise its heartbeat decides — announced, never silent. The id is per HOME, so two OS logins on one Mac are two machines. And a pid that is the asker's own no longer vouches for a different session's file: one Claude Code process hosts successive sessions.

One limit, stated plainly: an exclusive create is atomic on one filesystem. Two machines writing one project through a synced folder can both take a slot, and the sync resolves it as a conflicted copy. The lint check is what catches that; this is not distributed consensus.

Installed, and actually wired (0.9.13)

The session-start component check answers two questions, not one:

Question	How
Are the right files here, unmodified?	every shipped file is hashed into `MANIFEST.json` at install time and compared
Is any of it connected to anything?	`MANIFEST.json` also declares the hook entries the plugin expects in `settings.json` — fourteen for gt itself as of 0.17.2, plus any an installed module declares — and the check compares those too

Before 0.9.13 only the first question was asked, so a machine with every file installed and an empty `settings.json` reported **components: clean** while nothing ran. The clean line now reads *"installed matches , all N hooks wired"* — the second clause is the part that was missing.

Two states are reported: `unwired` (no entry for that event names the script — it never runs) and `badpath` (wired, but the command names a path that does not exist on this machine, which is what a deleted version directory or an unquoted path containing a space produces).

The declaration lives in `gt_components.HOOK_REGISTRATIONS` and is what `install.sh` registers *from*, so the installer and the checker cannot disagree about what "wired" means. `install.sh` verifies its own eight entries immediately after writing them (the five `SessionStart` reporters — `gt_surface.py` joined them in 0.17.2 — `gt_state.py` on `UserPromptSubmit` and `PreCompact`, and `guard_protected_paths.sh`); the six enforcement hooks are owned by `vault_init.py install-core-rules`, which needs a vault. `selftest.sh` asserts every declared entry from outside — the only vantage point that still works when nothing is wired at all.

Security: gt unlock (0.20.1)

Off by default. On, the unlock authority (`gt_unlockd.py`) identifies every caller through the kernel, collects your factors itself, and grants a session for 15 idle minutes / 8 hours.

Command	Purpose
`gt_unlock.py enroll touchid|hello|totp|sso` , `recovery`	Add factors; print one-time recovery codes
`gt_unlock.py policy enable|disable|show|approve|consent|unattended|sso`	Change the policy (needs your factors)

Command	Purpose
`gt_unlock.py unlock`, `lock`, `status`, `verify`, `audit`	Grant, revoke, show the level, self-check, read the audit log
`gt_unlock.py seal …`, `secret get`, `run`, `git-credential`, `aws-credential`	Sealed and brokered credentials, released only under a grant
`gt_lock.py add|open|restore|status`	Per-file age locks for chosen vault folders (never `core-rules/`, `global-memory/`)
`gt_unlock_touchid.py build`, `gt_unlock_hello.py available|selftest`	Build the Touch ID helper; check Windows Hello

Levels: off (default), L1 (TOTP/SSO: stops agents and accidents, not malware running as you), L2 (Touch ID / Hello in every unlock, credentials sealed). *"Unlock proves a person was present and limits what agents can do on their own. It is not anti-malware. If something already runs as you, it can wait for you to unlock."* Setting: `unlock`. Doctor rows: `unlock`, `security`.

Security: gt sandbox mode (0.20.1)

Off by default. On, Claude Code's own sandbox (macOS, Linux, WSL2) and permission rules keep the assistant's shell and file tools off the vault and gt's state; the vault is reached through gt's MCP server and the write queue. Native Windows: permission rules only (friction).

Command / tool	Purpose
`gt_settings.py set sandbox_mode on|off`	Write / remove the Claude Code settings (restart Claude Code; switch off from a terminal)

Command / tool	Purpose
`gt_sandbox.py status|plan|apply|remove|check|verify|managed|rollback-check`	Show, preview, re-apply (only while `sandbox_mode` is on), take out, check drift and loosening keys, self-check, print the admin-required config, refuse a rollback that could not remove them. Sandbox mode is a preview in 0.20.1
gt-vault MCP: `vault_search`, `vault_read`, `vault_list`	Read the vault (read-only, capped, never a locked folder)
gt-vault MCP: `vault_queue_write`, `vault_queue_drain`	Queue a write and get the broker's decision; apply what `gt_write_queue.py` left in `~/.gt-inbox/queue/`
gt-vault MCP: `vault_report`, `vault_task`, `vault_tasks_regen`, `vault_log_add`, `vault_events_emit`, `vault_adr`, `vault_project`, `vault_handoff`, `vault_closeout`, `vault_derived_write`, `vault_source_store`, `vault_optimize` (0.20.2)	A skill's vault scripts, run outside the sandbox with the same checks — every skill has its route
gt-vault MCP: no tool writes `design.md` or `global-memory/` (0.20.2)	They are changed by you, in an editor; a refused target costs no unlock prompt

Settings: `sandbox_mode`, `sandbox_vault_reads`, `vault_mcp`. Doctor row: `sandbox`; verify rows `sandbox-*`. `SECURITY.md` §8.

Security: a fingerprint for pushes and commits (0.20.4)

Off by default; three settings for the repos named in `push_fingerprint_repos`, on github.com or GitHub Enterprise (the host comes from each repo's remote).

Setting / command	Purpose
`push_fingerprint on|off`	A pre-push hook asks gt unlock for `gt:publish`: one fresh Touch ID (macOS) or Windows Hello per push. An unlock policy already on keeps every scope; only push, tag and the policy itself become step-up
`push_fingerprint_seal_token on|off`	The HTTPS push token is sealed behind `gt:publish`, so a plain `git push --no-verify` still needs the touch (not a helper the caller sets itself while gh is signed in); a process from Claude's shell is refused the token before any prompt
`commit_fingerprint on|off` (macOS)	A Secure Enclave key signs every commit and tag; one Touch ID each
`gt_sign.py github [--apply|--remove]`	The server half: the signing key and a ruleset requiring signed commits on the default branch, no bypass actors
`gt_push_guard.py check`, `gt_sign.py check --live`	Proof (exit 0 set up, 1 broken; `check` exits 2 when the guard is off)

`off` undoes each one exactly; the signing key stays registered so past commits stay Verified. Windows Hello commit signing is planned for 0.20.5. `SECURITY.md` §5.5a and §5.5b.

Checks, Cadences and Scheduled Jobs

Scripts with their own command line, shipped with the plugin at `~/.claude/plugins/cache/golden-thread-plugin/gt/<version>/scripts/` rather than in the vault — a git hook and a launchd job must find them with no vault open. The hook resolves the newest installed release by glob, so upgrading does not rewrite it.

The three cadences

The same scanners, run at three widths. The width is what keeps each one usable.

Cadence	Scope	Posture
`tests/run.sh`	the file set the scanners define	fails the run
commit gate (`.githooks/pre-commit`)	the staged diff only	denies the commit
`gt_sweep.py --vault V [--path P] [--only secrets,code] [--exclude GLOB] [--json]`	the whole tree, weekly	**reports, never blocks**

A gate that scanned the whole tree would punish you for someone else's old code, so it reads only what you are adding — which means nothing re-examines what is already there. The sweep is that answer, and because it surfaces old debt it must never block. It files each run through `gt_check_report.py` and exits non-zero only when a member could not run.

The two checks

Command	What it does
`gt_secrets.py <path> [--staged] [--json] [--exclude GLOB] [--baseline F] [--write-baseline F]`	Credential scanner. **Nothing in the process ever prints matched source text, and no other check runs beside it** — a finding is `path:line`, a rule id and a LENGTH; never a prefix, never a redaction, never a hash (a hash of a short secret is crackable and still confirms a guess). That output contract is why it is *not* a `gt-scan` member: the 0.16.0 attempt leaked because a `naming` check printed raw source in the same run. `--staged` reads staged blobs, which are what the commit will contain. Exit 0 clean / 1 found / 2 could not run — and for credentials, *could not run* blocks.
`gt_scan_code.py <path> [--staged] [--sarif FILE] [--json] [--baseline F] [--write-baseline F] [--rules]`	Source validation against the `lint` rules in effect — the second `gt-scan` member alongside `gt_scan_language.py` ; `gt_scan.py --list` shows both. Rules are DATA from `lint` packs, in a documented subset of ast-grep's rule schema; gt evaluates that data and never executes anything a pack supplies. Tiers `text` and `stdlib` are always present, `astgrep` and `treesitter` only if the optional dependency imports, and **a rule whose tier is absent is reported SKIPPED, never silently passed** (exit 3: nothing found, but the scan does not cover what it was asked to). Emits SARIF 2.1.0. It

Command	What it does
	DOES print source text, because here the text is the finding — which is why it may never share a process with `gt_secrets`.

Accepted findings go in a baseline (`.gt/secrets-baseline.json` , `.gt/code-baseline.json`), which the gate and the sweep pick up automatically; new findings still fire. Entries are content-keyed since 0.20.1 (rule, version-normalised path, a hash of the line -- salted PBKDF2 for secrets, no text stored), so a version cut does not re-flag them. The gate's two escapes are loud: `git commit --no-verify` once, or `git config gt.secretsgate off` for the clone, which prints that it is off on every commit.

The record

Command	What it does
`gt_check_report.py record --vault V --check <name> --verdict clean|findings|cannot-run [--count N] [--scope "…"] [--ref R] [--detail T]` · `show --vault V [--check C] [--json]`	Files a check's result into the vault so a gate leaves a record: a verdict, a count, a scope and a ref, **never a finding's content** — a vault file is committed and pushed, and a report naming `path:line` for a credential would republish every secret's location. Answers what an exit code cannot: is this check running, is it getting noisier, did anyone look. `cannot-run` is a first-class verdict, never laundered into `clean` . `record` always exits 0 and never commits.

Code review — the framework, with no opinions in it

Command	What it does
`gt_code_review.py dimensions [--vault V]` · `plan <path> [--staged]` · `validate <findings.json> --root P` · `report <findings.json> --root P --vault V [--ledger F]`	The deterministic half of code review. gt does not judge — *"does this abstraction earn its keep"* has no mechanical oracle — it plans which dimensions over which files, then rejects mechanically any finding citing a file that does not exist, a line out of range, an unknown dimension, an invalid severity or `confirmed: false` , and

Command	What it does
	only survivors reach the record. `--ledger` keeps a previously declined finding from returning as new.

gt ships ZERO review dimensions, deliberately. The `review` slot is empty by design: the opinions are the user's or the company's, supplied as a `review.*.pack.json` in their own vault, where they get precedence, shadowed reporting, retraction and provenance from the same registry as everything else. **Zero dimensions configured exits 3 and says how to add one — it is never reported as a clean review**, because "nothing was reviewed" and "reviewed, nothing found" are different facts and only one is reassuring.

The scheduler

Command	What it does
`gt_schedule.py list` · `install <job> --vault V [--repo PATH …] [--hour H] [--minute M]` · `check <job>` · `remove <job>`	Installs, verifies and removes gt's launchd jobs (`daily` 22:00, `lint-weekly` Mondays 07:00, and since 0.17.2 `sweep` Mondays 07:30 — whose pre-flight currently refuses, because the registry cannot find its packs from the hooks dir). A job runs the copy installed in `~/.claude/golden-thread/hooks/` ; since 0.17.2 every job's script is installed there, and the release gate fails one that is not. Normal exits are only codes a script returns on purpose — `lint-weekly` listed `1` as normal until 0.17.2, and `1` was only ever its crash; a crashed lint now exits 3 and says COULD NOT RUN. Exists because the weekly lint agent was installed by hand on 2026-09-08 with no `--check` and no rollback. **Validation goes through launchd, not a terminal run:** `install` bootstraps the job, kickstarts it, waits, then reads launchd's own last exit code and the job's output — a job that works in a terminal can still fail under launchd. `remove` boots out and deletes the plist, which is what makes `install` safe to re-run. **Known, not fixed:** macOS refuses calendar-fired runs access to a vault under CloudStorage (every calendar-fired `lint-weekly` run on record); `/gt:gt-doctor` 's `schedule` check reports it.

Writing to the vault: the queue (Core rule 1, 0.17.11)

Command	What it does
`gt_write_queue.py --vault V --path REL --op append|replace-section|create|set-property|replace-file (--content T | --content-file F | --value V) [--section H] [--key K] [--session ID] [--origin session|farm] [--hint H] [--dry-run] [--json]`	Asks for a vault write instead of making it; the request never touches its target. `set-property` (new in 0.17.11) sets one top-level frontmatter key from `--key` / `--value`; `replace-file` (new) replaces the whole file, guarded by the hash of what was read. A farmed result (`--origin farm`) is only ever applied automatically as `create` . Installed into the hooks dir since 0.17.11, so scheduled jobs and the vault's tools can reach it.
`gt_broker.py drain --vault V [--dry-run] [--json]`	Applies every queued write once, oldest first, then exits — no daemon, no hook. Appends merge and near-duplicates drop. A write to a file another live session has claimed is **held** (still queued, applied by a later drain). A write whose target changed after it was queued, or was moved or deleted since, conflicting replacements, and **every** write to a `design.md` or `global-memory/` are **escalated**: nothing is written, every version is kept in `spool/broker/conflicts/` , and a `#conflict` task goes to the owner. It never recreates a file at its old path.
`gt_broker.py status --vault V` · `audit --vault V [--since HOURS]`	`status` : how many writes wait (read-only; also shown at session start as "WRITE QUEUE: N waiting"). `audit` (new in 0.17.11): vault `.md` files changed in the window (default 24 h) that the broker did not write — a report, not an alarm; the owner's own Obsidian edits appear too.

Since 0.17.11 `gt_daily` , `gt_task` , `gt_lint_weekly` , `gt_handoff` , `gt_handoff_status` and `wiki_log` route their vault writes through the queue. `log.md` , `decisions.md` and `TASKS.md` keep their own tools (`gt_log.py add` , `gt_adr.py allocate` / `merge` , `gt_tasks.py`). Not queue ops: deleting or moving a file (`gt_demote.py` still does it directly).

Session capture

Command	What it does
`gt_daily.py --vault V [--date YYYY-MM-DD] [--repo PATH …] [--dry-run] [--check]` · `finalize --vault V (--date D | --before D | --all-past) [--dry-run [--diff]]`	Writes the day's FACTS into `Daily Notes/<date>.md`: tasks closed, commits per repo, event counts, wiki item counts, an active span per project. Terse by design — it does not explain, summarise or interpret, because a generated block that editorialised would encode a reading of the day that is not the owner's. Each fact goes under its heading in a marked block, replaced whole each run (`## Did`: new projects, tasks closed, commits; `## Decided`: ADRs added; `## Open at end of day`: tasks added and due), counts in a footer block; the owner's own lines stay above the blocks, and it never touches `## Noticed`, the unfiled capture surface `gt-review` sweeps. Git is primary and events are enrichment: on 2026-09-27 the event log held 5 events on a day with 10 commits across three work streams. **Finalize** (0.21.4, setting `daily_finalize`, default on): the first run on a later day writes each of the last seven days' notes in its final form — every comment on its own lines (template guidance, gt_daily's markers) removed with the generated content kept as plain text, empty `-` / `- [ ]` placeholders and empty template headings dropped, marked `gt_daily: final` so no later run writes it again; the owner's lines, open `## Noticed` checkboxes included, are kept exactly, and today's note is never touched. `finalize` does a backlog; `--dry-run` prints what each note would lose.
`gt_state.py check [--margin N] [--write] [--json] [--session ID]` · `write [--reason R] [--session ID]` · `show` · `hook`	Writes session state BEFORE the context runs out. **The signal is `ctx_pct`, never `rate_limits`**. Since 0.18.1 a hook run tells **you** through `systemMessage` — a line when the threshold write happens or fails, and always one at `PreCompact` — and reads only this session's own ledger rows (by `session_id`), so a new session no longer reports the previous one's context figure.

Command	What it does
`gt_surface.py check [--vault V] [--dry-run] [--json]` · `must-do [--vault V]` · `handoffs --project SLUG [--vault V]`	**New in 0.17.2, a `SessionStart` hook** (setting `surface`, default on) — the reader for what the tools above write for the next session, which until then nothing at session start read. Every session: a **MUST DO** block from `<vault>/deadlines.md` (one table, `| item | category | due | see |`, `due` as `YYYY-MM-DD`), overdue 🔴 or due within 14 days 🟡, later rows counted, computed live from the date; a malformed row is skipped and counted, a credential-shaped label withheld. Every session until handled or deferred to a date: each waiting handoff (`handoff/*.md`, or `handoff*.md` beside a project README), one line — path, project, age, open items — never the body; setting `handoff_surface` picks where (`any` default, `project` only via `/gt:gt-open`, `manual` only via `/gt:gt-handle handoff`). Every session while any wait: one line counting `p:: 1` tasks waiting on you, overdue and open over a week (setting `task_surface`, counted by the vault's `gt_task.py`). Once each: a `gt_state.py` state file — handed to the model in full right after a compaction. **Surfacing is the alarm; the task is the record**: it never creates or closes a task and never writes the vault, only the machine-local `~/.claude/golden-thread/surface/seen.json` for state files. A clean start prints "nothing waiting"; every failure exits 0.
`<vault>/Projects/golden-thread/tools/gt_task.py add TEXT --vault V (--project SLUG | --inbox) [--ref REF] [--p N] [--waiting W] [--due D] [--dry-run]` · `list --vault V [FILTER …] [--json]` · `done ID [--reason T]` · `drop ID --reason T` · `defer ID --until D --reason T` · `count [--json]`	Vault tool (0.17.2) behind the three task skills. One store — README `## Tasks` — parsed with `gt_tasks.py`'s own rules, so a task written here ranks exactly like a hand-written one. New optional fields `[ref:: …]` and `[defer:: YYYY-MM-DD]`; `gt_tasks.py` hides a deferred task from ranking and escalation until its date. Done/drop check the box and append what settled it — a dropped task reads as closed to `gt_events` and `gt_daily`. Exit 1 when refused (a README claimed by another live session, or a stale ID).

Command	What it does
`gt_handoff_status.py list --vault V [--project SLUG] [--all] [--json]` · `mark FILE --vault V --status handled|open|deferred [--until YYYY-MM-DD] [--reason TEXT] [--dry-run]`	A handoff's state (0.17.2): `open` keeps surfacing; `deferred` needs a future `--until` and is open again on that date ("later, some time" is `handled` with a reason); `handled` is marked, or follows once every task citing the handoff's filename is checked off; `history` is a pre-0.17.2 handoff with no status, over a week old, no open citing task. `gt_handoff.py` now writes `status: open` in frontmatter. `mark` appends to the handoff's status log. Never loads a body.

Added in 0.18.1

Command	What it does
`gt_close.py project SLUG --vault V [--archive [--reason R] [--move]] [--json] [--dry-run]` · `task ID --vault V [--reason R]` · `handoff FILE --vault V --reason R`	The tool behind `/gt:gt-close`. `project` exits 1 while any task or handoff is undecided; `--archive` runs `vault_init.py archive-project` (in place); `--move` relocates to `Archive/<slug>/` through the queue. `handoff` refuses while a citing task is open. Also new: `gt_task.py shelve ID --reason R` (`p:: 7`) and `gt_task.py move ID --to SLUG --reason R`.
`<vault>/…/tools/gt_adr.py allocate <project> --title T [--supersedes N] [--expires-when C] [--expires D]` · `lineage <project> "<topic>"`	ADR supersession and expiry fields; the supersession chain of ADRs mentioning a topic (read-only). A bare sub-project slug resolves to its folder (`gt_spool.resolve_project`); unknown or ambiguous is exit 2.
`gt_entities.py --vault V lookup NAME --project S` · `list --project S`	Memory notes by their declared `entities:` (read-only).
`gt_brief.py --vault V SLUG [--repo PATH] [--dry-run]`	Draft a repo `CLAUDE.md` section to stdout; writes nothing.
`gt_catchup.py --vault V --project S [--brief | --no-brief] [--mark] [--json] [--dry-run]`	The catch-up paragraph `/gt:gt-open` leads with after time away (setting `brief_absence_days`).

Command	What it does
`gt_digest.py write --vault V --project S [--dry-run] [--json]` · `check …`	`research-digest.md`: newest 20 entries plus `[pinned]`, one line each, through the queue; `check` exits 1 when missing or stale.
`gt_memory_check.py --vault V --project S [--changed NOTE …] [--work] [--json]`	Contradictions between this session's memory notes and their neighbours (read-only; setting `memory_contradiction_check`).
`gt_promote_detect.py --vault V [--project S] [--threshold PCT] [--work] [--json]`	Promotion candidates: churned memory notes, cross-project research overlap, global scope leaks (read-only; settings `promotion_candidates`, `promotion_overlap`).
`gt_optimize_session.py [--days N] [--brief] [--check [PCT]] [--json]`	gt-optimize's `session` member: prompt-cache writes by cause, avoidable share, list-price equivalents (settings `optimize_session_days`, `optimize_avoidable_pct`).
`gt_minimize.py [--session ID | --transcript F | --latest] [--json]`	This session's billed context, estimated breakdown and cache warmth (read-only).
`gt_handoff.py … [--session ID] [--since-commit SHA] [--dry-run]`	Adds `## What Changed This Session`, measured from the `start_commit` `gt_session.py register` records.
`guard_foreign_checkout.py list` · `add PATH [--label L] [--route R] [--dry-run]` · `remove PATH [--dry-run]` (hooks dir)	Declare a checkout another machine owns; the `PreToolUse` guard `guard_foreign_checkout.sh` then denies `git commit` / `git push` inside it (setting `foreign_checkout_guard`).
`log_knowledge_read.sh` (hook, `PostToolUse` matcher `Read`)	Appends one line per Read of a Knowledge page to `<vault>/usage/knowledge.jsonl` (git-ignored; setting `knowledge_access_log`).
`gt_check.py list [--for PATH] [--json]` · `run [PATH …] [--staged] [--event commit-msg --message-file F] [--json] [--no-receipt] [--dry-run]`	The validation host: runs every module checker that applies; `pass` / `fail` / `cannot-check`, and `cannot-check` never passes. Exit 1 on a fail or cannot-check, 3 when nothing applied. Receipts feed the commit gate when `commit_checks` is on.

Command	What it does
`gt_apply.py list [--all]` · `show ID` · `apply (ID … | --all) [--dry-run]` · `undo ID`	The only writer of a checker's fix proposals: grant, protected paths, one in-place edit, content rules, claims, a re-check by every checker, compare-and-swap write; first-party only for auto-apply (`addon_fixes`, `addon_fix_size_limit`).
`gt_model.py resolve [INTENT]` · `skill PATH` · `check PATH …`	What `model_intent` (fast/balanced/deep) resolves to here, and from which pack; refuses unknown values.
`gt_sync.py status · pull · push · behind` (hooks dir)	The script behind `/gt:gt-sync`; `behind` prints the session-start line.
`gt_reminder.py status · setup CH · check CH [--json] · mirror --vault V · run [--scheduled] · import-tsv`	Reminders through macOS notification, the notification relay (SMS/Discord) or email, each off by default; `check` sends a real test; the `reminder` scheduled job reads a mirror, never the vault.
`gt_checkpoint.py find --tool scan|ingest|scan-language` · `show FILE` · `prune [--days 7]`	Resumable batch checkpoints, readable by any session; pruned after 7 days.
`gt_link_suggest.py suggest --page P` · `apply --page P --to T … [--forward-only]`	Up to five unlinked Knowledge pages, cross-domain first; writes only the picked links, both ways, through the queue.
`gt_review_stamp.py --vault V PAGE … [--date D] [--dry-run] [--json]`	Queue `last_reviewed` on Knowledge pages gt-query and gt-open read (`review_stamp`); gt-wiki 0.2.5's `review-due` ages from it.
`gt_pipeline.py init · list · check · add · set · remove · render · run · flag` (`--repo R`)	A project's release pipeline: `release-pipeline.tsv` (steps and gates as data) and a generated `release.sh` that stops at the first failure and says what ran. Defaults `@tests @allin @branch @install @owner-gate @push @sync`; a default gate is removed only with `--reason`. `flag` sets README `release_pipeline:`.

Command	What it does
`gt_recipe.py render · check · show`	Recipe (`*.tsv` steps table or `*.recipe`) → generated `.sh` through one tested template; `check` fails on a hand edit.
`gt_metrics.py record · time · report · peers · mark · verify`	One row per execution, per project and machine (`execution_metrics`); rolling baselines, regressions with what changed, cross-project peers, before/after of an applied change.
`gt_optimize_exec.py` = `gt_optimize.py --only execution` (alias `--execution`)	gt-optimize's opt-in third member: execution findings ranked by measured cost; `--apply ID` after a yes.
`gt_bench.py [--hosts] [--dry-run]` · `health`	Measure `parallel_profile` (the throughput knee) instead of 2× cores; `health` = doctor's `execution` row (measured or default, Rosetta, TMPDIR in a synced folder).
`gt_load.py [--cap N] [--json]`	What a parallel run would start with now: the ceiling scaled by load and memory pressure.
`gt_ingest_pipeline.py stages · survey · packet · fan-in · reconcile · draft · promote-scan · promote-plan · status`	The deterministic stages of ingest (intake-scan → survey → extract → classify → reconcile → draft) and promote (verify → generalize → place → owner); stops only on a contradiction, a security issue or unsafe code; no command applies a promotion plan.
`tests/run.sh --affected` · `prun.py --hosts a,b`	A scoped run and receipt (accepted on feature branches, `scoped_receipts`); units split across the `runners` .

gt-wiki Skills (5)

The gt-wiki plugin provides an LLM-powered knowledge base separate from the Golden Thread memory hierarchy. Sources are ingested immutably; Knowledge pages are synthesized

summaries linked bidirectionally.

Key design decision: The vault `CLAUDE.md` is kept slim (architecture + pointer only). Page format schema lives in `Knowledge/_template.md` and is read on demand by the ingest and refresh skills — it is not loaded into every session's context.

Command	What it does
`/gt-wiki:gt-wiki-init`	Set up a new wiki vault from scratch. Runs `vault_init.py` deterministically — creates `Sources/`, `Knowledge/`, `index.md`, `log.md`, seeds `CLAUDE.md` from template, and stamps `Knowledge/_template.md`.
`/gt-wiki:gt-wiki`	Query the wiki. Reads `index.md` first, follows wikilinks, falls back to grep, deep-digs Sources for precision. Logs every query.
`/gt-wiki:gt-wiki-ingest`	Add a source: fetch or paste, store immutably in `Sources/`, discuss with user, write Knowledge pages per `_template.md`, cross-link bidirectionally, then update `index.md` and `log.md` through `wiki_log.py` so every entry has the same shape. Records `upstream_sha:` for sources inside a git repo.
`/gt-wiki:gt-wiki-lint`	Run `wiki_lint.py` (10 deterministic checks). Interprets findings, proposes fixes, records declines in `lint-declines.md` so nothing gets re-litigated.
`/gt-wiki:gt-wiki-refresh`	Check selected sources for upstream changes. `wiki_refresh.py` detects changes deterministically for local sources (`git fetch` + `git diff` from the source's `upstream_sha:`, or its `ingested:` date) and flags web-only sources for the LLM to fetch and compare. Supersedes changed ones with new immutable source files. Updates citing Knowledge pages.

Vault Structure

```
<vault>/
  CLAUDE.md                  ← architecture + pointer to Knowledge/_template.md
  index.md                   ← navigational index of all Knowledge pages
  log.md                     ← audit trail: GENERATED from the per-session spools
  INBOX.md                   ← the capture point: one checkbox line, any session
  TASKS.md                   ← GENERATED cross-project task rollup (gt_tasks.py)
  review-queue.md            ← items flagged for owner review
```

```
lint-declines.md            ← lint findings you declined, so they are not re-litigated

Sources/                    ← IMMUTABLE raw originals
  YYYY-MM-DD <title>.md

Knowledge/                  ← cross-project wiki pages
  _template.md              ← page format schema (read by ingest/refresh skills)
  <Page Title>.md

global-memory/
  MEMORY.md                 ← index; loaded every session
  <topic>.md

Projects/
  README.md                 ← master project list
  CONVENTIONS.md            ← lifecycle phases, file roles, tag taxonomy
  PROTOCOL.md               ← cross-project process rules
  INFRASTRUCTURE.md         ← server fleet, defined once

  <project-slug>/
    README.md               ← status board + YAML frontmatter
    source.md               ← where code lives, topology, deploy file plan
    idea.md                 ← original brain dump — IMMUTABLE
    research.md             ← append-only findings
    decisions.md            ← append-only ADRs
    design.md               ← current architecture
    spec.md                 ← handoff artifact (created when design is complete)
    runbook.md              ← operational procedures (optional)
    memory/
      MEMORY.md             ← session memory index
      <topic>.md

  golden-thread/
    core-rules/             ← canonical Core rule definitions
    tools/                  ← gt_log.py, gt_adr.py, gt_spool.py, gt_tasks.py,
                              gt_session.py, gt_closeout.py, gt_edits.py,
                              gt_events.py, safe_write.py
    spool/                  ← per-session log and ADR spools (log.md/decisions.md
                              are rendered from these, never written directly)
```

Immutability Model

File	Mutability	Rule
`Sources/*.md`	Immutable	Never edited; superseded by new files
`idea.md`	Immutable	Origin story; never changed
`research.md`	Append-only	History is extended, never rewritten
`decisions.md`	Append-only	Reversed by adding a new ADR, never by editing
`design.md`	Mutable	Always describes the current architecture
`spec.md`	Mutable (scope only)	Self-contained handoff doc
`runbook.md`	Mutable	Operational HOW-TO; incubator before CLAUDE.md
`PROTOCOL.md`	Append-only	Cross-project process rules
`Knowledge/*.md`	Mutable	Synthesized summaries; cite `Sources/`
`core-rules/*.md`	Mutable	Editing here changes what hooks inject

Context Footprint

> **Unverified — undated.** The token figures below carry no date and no provenance, and nothing in the source tree derives or checks them; the 2026-09-17 accuracy pass could not establish which release they were measured against. Read them as orders of magnitude, not as measurements, and re-measure before quoting them anywhere.

What loads	When	Approx tokens
`~/.claude/CLAUDE.md` (global instructions)	Every session	~200
`<vault>/CLAUDE.md` (architecture + pointer)	Every session in vault dir	~100

What loads	When	Approx tokens
`global-memory/MEMORY.md` (index only)	Every session	~50
`Knowledge/_template.md` (page schema)	Only during ingest/refresh	~150
Project memory files	On demand via `gt-open`	Varies

The slim CLAUDE.md design (introduced in gt-wiki 0.1.0) drops the vault's cold-start footprint from ~1,300 tokens to ~350.

Source Topology

Every project records where its code lives in `source.md`.

Topology	Meaning
`local`	A folder on this machine. No deploy step.
`remote`	One repo, one server; server pulls from GitHub.
`bastion-jump`	Several servers, all reached through one gateway host.
`bastion-direct`	Several servers, each reachable independently.

Bastion projects carry a **file plan** marking each deployed file `static` (byte-identical everywhere) or `unique` (per-server variant that must never be cross-deployed).

Install

```
# From zip
unzip golden-thread-plugin.zip
cd golden-thread-plugin
bash install.sh
# Restart Claude Code
```

Installs `gt` (v0.17.2) and each module that is on — `wiki`, `demo`, `watch`, `report-card` and `flow` on by default, `farm` off for a fresh install — as separate plugins under the `golden-thread-plugin` marketplace. Choose modules with `--list-modules`, `--without <name>` and `--with <name>` (remembered). Re-running upgrades from any older release. Requires Python 3.8+.

`install.sh` installs the **newest version directory** present, not a hardcoded constant — pass an argument only to roll back deliberately (`./install.sh 0.14.0`). Never pipe it to `head`: `set -o pipefail` turns the closed pipe into an abort partway through, leaving the cache updated and registration undone.

Typical Session Pattern

```
/gt:gt-open my-project        # load project, summarize state

# ... work happens ...

/gt:gt-route                  # "where is this going?" — run it when the session
                              # has drifted, or before writing something down and
                              # you are unsure which file it belongs in

/gt:gt-work                   # write back findings, update docs

# periodically
/gt:gt-lint                   # catch structural drift
/gt-wiki:gt-wiki-ingest <url> # add a source to the wiki
/gt:gt-promote                # graduate a finding to Knowledge or global-memory
/gt-flow:gt-flow              # see how knowledge moved (--redact before sharing)
/gt-visualize:gt-visualize    # explain a codebase in 3D, or see it as a code city
```

Requirements

- Python 3.8+
- Claude Code (any version)
- Obsidian (optional — the vault is plain markdown; Obsidian is the GUI layer)